

Nota Técnica — Hotel Fácil

Decisões de Arquitetura, Trade-offs e Melhorias Futuras

Contexto

O sistema foi desenvolvido como uma aplicação de reservas de hotéis com frontend em Angular 19 e backend em Spring Boot 3, com banco MySQL. O deploy foi feito no Railway (backend) e Vercel (frontend).

Esta nota descreve as principais escolhas técnicas feitas, o que elas implicam em termos de limitações, e o que eu mudaria ou evoluiria se o sistema fosse para produção de verdade.

Decisões e Trade-offs

Comunicação REST síncrona

Escolhi REST por ser o padrão mais simples e direto para um CRUD. Funciona bem no volume atual, mas operações como envio de e-mail de confirmação ou geração de relatórios ficam presas no mesmo ciclo do request. Em produção com volume maior, eu adicionaria um message broker (RabbitMQ ou Kafka) para desacoplar essas operações do fluxo principal.

JWT stateless

O token fica no cliente e o backend não guarda estado de sessão. Isso facilita muito escalar horizontalmente — qualquer instância consegue validar qualquer token sem precisar de sessão compartilhada.

O ponto fraco é a revogação: se um token for comprometido, ele continua válido até expirar. A solução que usaria é um blacklist no Redis com os tokens revogados, consultado a cada request.

Banco de dados único

Um MySQL centralizado resolve bem para o escopo do projeto. O problema é que em carga alta ele vira gargalo, principalmente nas leituras. A próxima evolução seria uma read replica para consultas e um cache Redis para dados que mudam pouco, como lista de hotéis e detalhes de quartos.

Monólito

Todo o backend está em um único serviço. Foi a escolha certa para o prazo, mas o acoplamento entre os domínios (hotéis, quartos, reservas, autenticação) dificulta evolução independente. Com mais tempo, separaria pelo menos o serviço de notificações, que não precisa estar no fluxo crítico.

Race condition nas reservas

Esse foi o problema mais interessante do projeto. A verificação de disponibilidade e a criação da reserva são dois passos separados, o que abre janela para double-booking em requisições concorrentes — documentado em detalhes no INCIDENT_ANALYSIS.

A correção mais direta seria um *SELECT FOR UPDATE* no momento de verificar disponibilidade, garantindo que a leitura e a escrita aconteçam dentro da mesma transação com lock. Como segunda camada, uma unique constraint composta (room_id, check_in, check_out) no banco barraria qualquer duplicata que escapasse.

O que eu evoluiria

Se o sistema fosse crescer, as mudanças em ordem de prioridade seriam:

1. **Lock pessimista e constraint no banco** para resolver o race condition de vez. Isso é crítico e deveria ser a primeira coisa a fazer.
2. **Redis** para cache de leituras frequentes (hotéis, quartos) e para blacklist de tokens revogados.
3. **Comunicação assíncrona** para operações de notificação e relatório,

desacoplando do fluxo principal via fila de mensagens.

4. **API Gateway centralizado** para autenticação, rate limiting e roteamento, preparando o terreno para uma eventual divisão em microsserviços.
5. **Escalabilidade horizontal** com múltiplas instâncias do backend atrás de um load balancer, o que já seria suportado pelo JWT stateless e pelo cache distribuído.

A decomposição em microsserviços só faria sentido com equipe maior e tráfego que justificasse a complexidade operacional.